

MANUAL TECNICO DEL SISTEMA WEB PARA LA GESTION DE LOS PROCESOS ADMINISTRATIVOS EN LAS JUNTAS CATONALES DE PROTECCION DE DERECHO

Índice

INTRODUCCION.....	3
Usuarios del sistema	3
Diagramas de flujo de las fases de una denuncia	3
ARQUITECTURA DEL SISTEMA.....	4
Stack Tecnológico Base.....	4
Arquitectura	4
Diagrama general de Arquitectura	4
ENTORNO DE DESARROLLO Y CONFIGURACION.....	4
Requisitos Previos del Sistema.....	4
Variables de Entorno (Configuración Local).....	6
Variables de Entorno (Configuración supabase)	7
DISEÑO DE LA BASE DE DATOS.....	8
Modelo entidad relación parte de usuarios e involucrados en la denuncia	8
Modelo entidad relación asignar medidas y vulneraciones a afectados	9
Modelo relación denuncia y sus diferentes fases.....	10
Diagrama relación fase audiencia de contestación	12
Diagrama relación fase audiencia de pruebas.....	12
Diagrama entidad relación fase cierre de caso	13
Diagrama fase cumplimiento de medidas denuncia	13
Diagrama relación subida de expedientes	14
BACKEND.....	14

Estructura del código base	14
Seguridad	15
Reglas de Negocio	16
Flujo de procesamiento y especificaciones de endpoints	17
FRONTEND	19
Estructura código base	19
Diagrama de jerarquía	21
Diagrama de flujos formulario	21

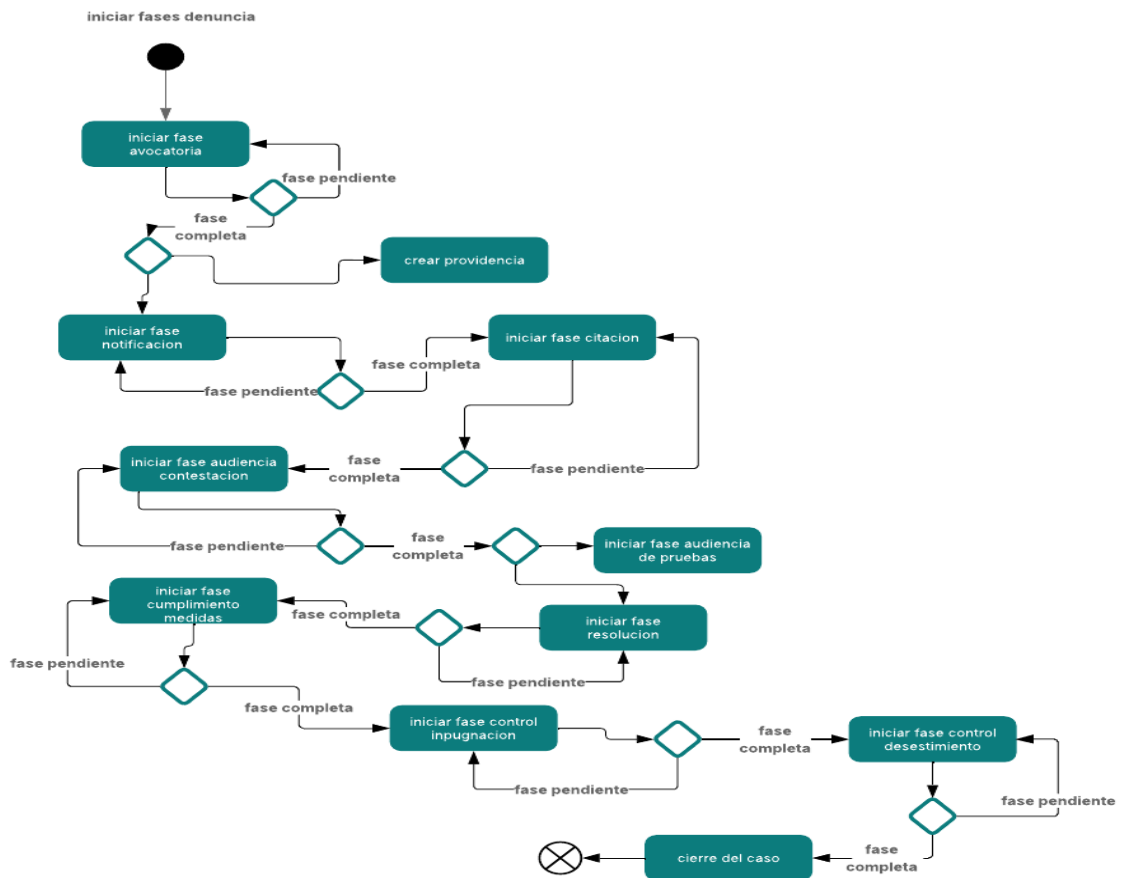
INTRODUCCION

El presente sistema web fue desarrollado para automatizar y gestionar los procesos administrativos de las Juntas Cantonales de Protección de Derechos (JCPD) en Manabí. Permite el control de expedientes, gestión de usuarios con diferentes roles y la generación de reportes estadísticos.

Usuarios del sistema

- Administrador: encargado de gestionar a los usuarios
- Miembros: encargados de llevar el control de los expedientes y visualizar reportes estadísticos: principal, suplente, secretaria.

Diagramas de flujo de las fases de una denuncia



ARQUITECTURA DEL SISTEMA

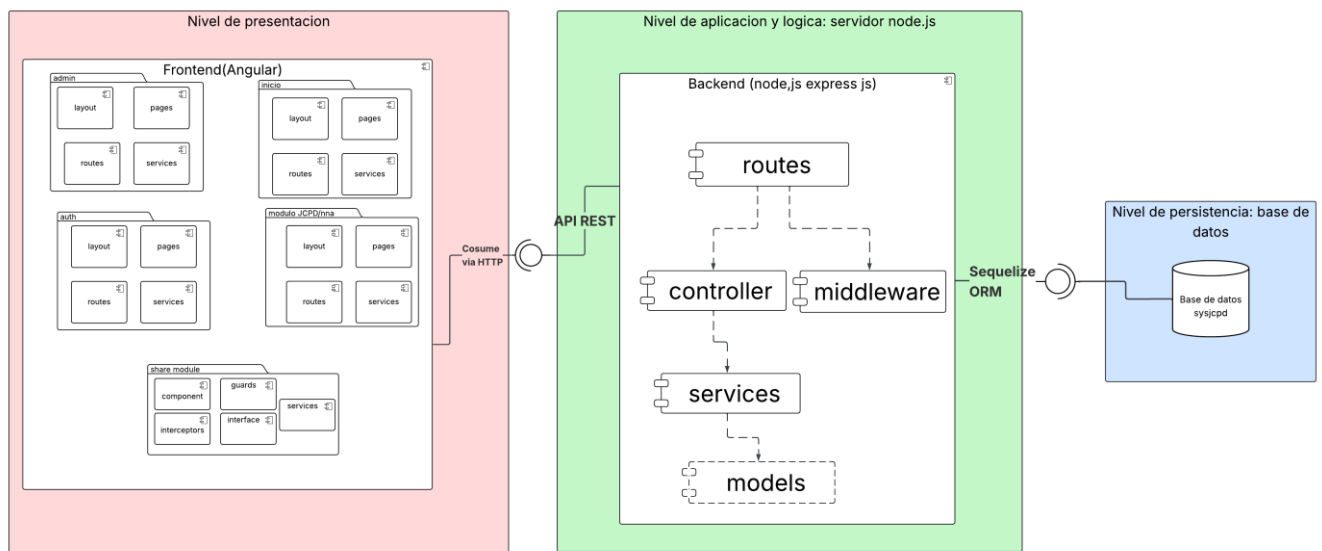
Stack Tecnológico Base

- **Frontend (Capa de Presentación):** Angular (v 19.2.19), TypeScript, HTML5, TailwindCSS.
- **Backend (Capa de Lógica de Negocio):** Node.js (v20.18.0), Express.js (v5.1.0), TypeScript.
- **Base de Datos (Capa de Persistencia):** PostgreSQL (v17).
- **ORM/Query Builder:** Sequelize (6.37.8).

Arquitectura

El sistema opera bajo una arquitectura Cliente-Servidor. La comunicación entre las capas se realiza mediante el intercambio de datos asíncronos en formato JSON a través del protocolo HTTP.

Diagrama general de Arquitectura



ENTORNO DE DESARROLLO Y CONFIGURACION

Requisitos Previos del Sistema

- Node.js v20.x o superior.
- Angular CLI (`npm install -g @angular/cli`).
- PostgreSQL (17).

Dependencias de Desarrollo

El servidor se desarrollo con el lenguaje de programación typescript por lo que para el desarrollo se usaron las siguientes dependencias

Categoría	Dependencia (Paquete)	Versión	Propósito Técnico en el Sistema
Compilación y Ejecución	typescript	^5.8.3	Transpilador principal que convierte el código fuente estricto a JavaScript nativo compatible con Node.js.
	ts-node-dev	^2.0.0	Motor de ejecución para el entorno local. Compila en memoria y reinicia el servidor automáticamente ante cualquier cambio en el código, agilizando el desarrollo.
Tipos Base	types/cors types/express types/fs-extra types/jsonwebtoken types/multer types/node types/pdfmake		Tipado estricto para la estructuración

	types/pg types/socket.io		
--	-----------------------------	--	--

Variables de Entorno (Configuración Local)

Variable	Descripción	Ejemplo
DB_HOST	Servidor de base de datos	localhost
DB_USER	Usuario con privilegios de la base de datos	postgres
DB_PASSWORD	Contraseña de PostgreSQL	tu_password
DB_DATABASE	Nombre de la base de datos	jcpd_db_dev
JWT_SECRET	Semilla criptográfica para tokens	clave-secreta-local
PORT	Puerto de ejecución de la API	3000
URL	URL permitida del cliente (Angular) para establecer la conexión bidireccional por WebSockets	http://localhost:4200
STORAGE_TYPE	El tipo del almacenamiento si será 'local' o 'cloud al subir archivos	local

Variables de Entorno (Configuración supabase)

Variable	Descripción	Ejemplo
DB_HOST	Servidor de base de datos	db.urlejemplo.supabase.co
SUPABASE_KEY	Clave de acceso (API Key) generada por Supabase para autenticar las peticiones del backend hacia sus servicios externos.	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6bnZsanVpZGR0ZXp6dWdyZ2pma3R4Iiwicm9sZSI6InNlcnZpY2Vfcm9sZSIsImhhdCI6MTc3MzZmZDZlUxM
SUPABASE_URL	Endpoint o URL principal del proyecto en Supabase. Necesario para inicializar el cliente y consumir sus servicios en la nube (como el almacenamiento de archivos o <i>Storage</i>).	https://vljuiddtezzugrgjftx.supabase.co/
DB_URL	Cadena de conexión completa (<i>Connection String</i>). Utilizada por el ORM (Sequelize) para establecer la comunicación directa con el motor de base de datos.	postgresql://postgres.vljuiddtezzugrgjftx:tu_password@aws-1-us-east-1.pooler.supabase.com:6543/postgres
JWT_SECRET	Semilla criptográfica para tokens	clave-secreta-local
PORT	Puerto de ejecución de la API	3000
URL	URL permitida del cliente (Angular) para establecer	http://localhost:4200

	la conexión bidireccional por WebSockets	
STORAGE_TYPE	El tipo del almacenamiento si será 'local' o 'cloud al subir archivos	cloud

DISEÑO DE LA BASE DE DATOS

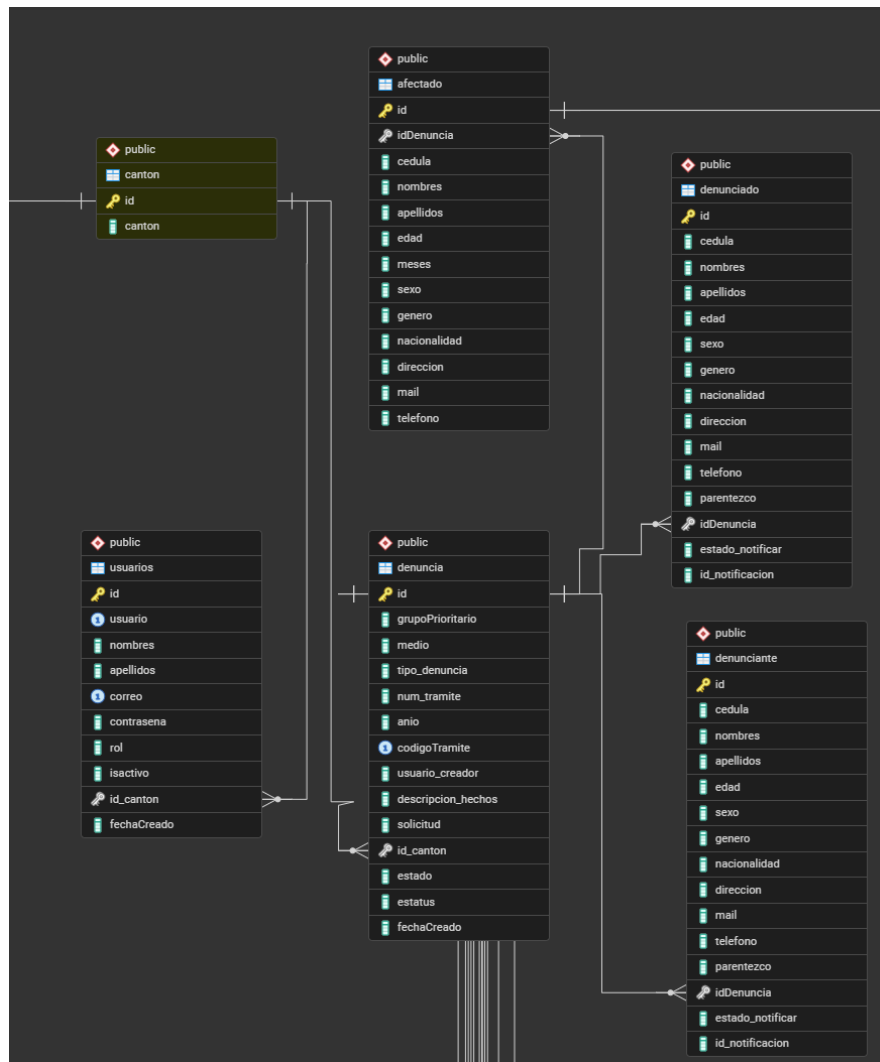
A continuación, se presentarán por partes el diagrama entidad para una mejor representación de las relaciones entre tablas se dividirá de la siguiente manera:

Usuarios y partes involucradas, asignar medidas y vulneraciones a afectados, denuncia y sus diferentes fases, fase audiencia de contestación, fase audiencia de prueba, fase cierre de caso, fase cumplimiento de medidas, subir expedientes

Modelo entidad relación parte de usuarios e involucrados en la denuncia

El siguiente diagrama muestra la relación que existe entre la tabla denuncia y los diferentes datos de los denunciados denunciantes, y afectados.

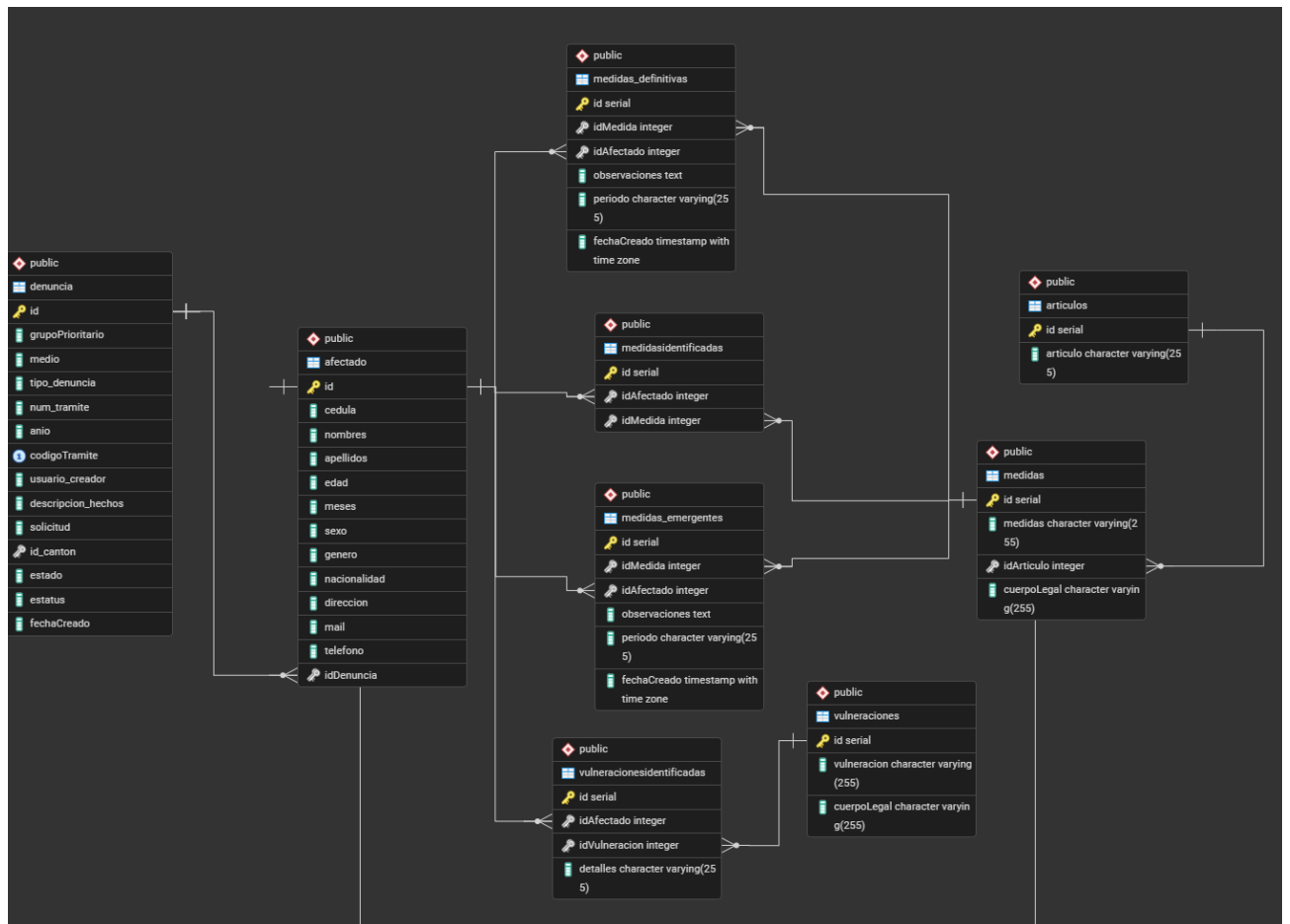
Además, muestra la relación de denuncias y usuarios que pertenezca a un cantón



Modelo entidad relación asignar medidas y vulneraciones a afectados

Esta relación muestra la asignación de sus medidas en los distintos procesos ya sea cuando recién se las identifica, cuando ya pasan a emergentes y cuando son definitivas

Además, también cuando se identifican los vulneraciones



Modelo relación denuncia y sus diferentes fases

Aquí se visualiza la relación como la denuncia es central y de ella esta relacionada y depende todas las demás fases

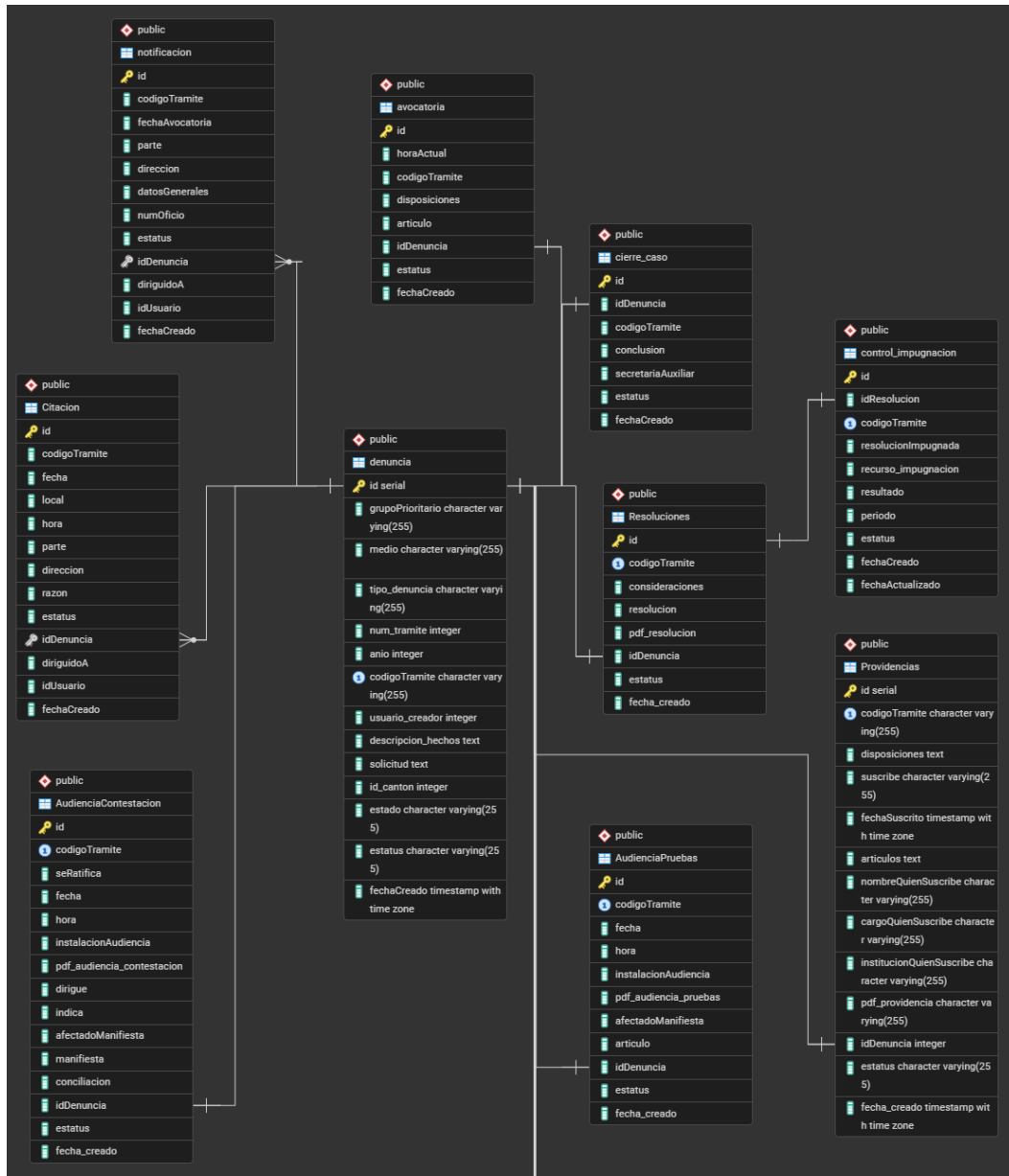


Diagrama relación fase audiencia de contestación

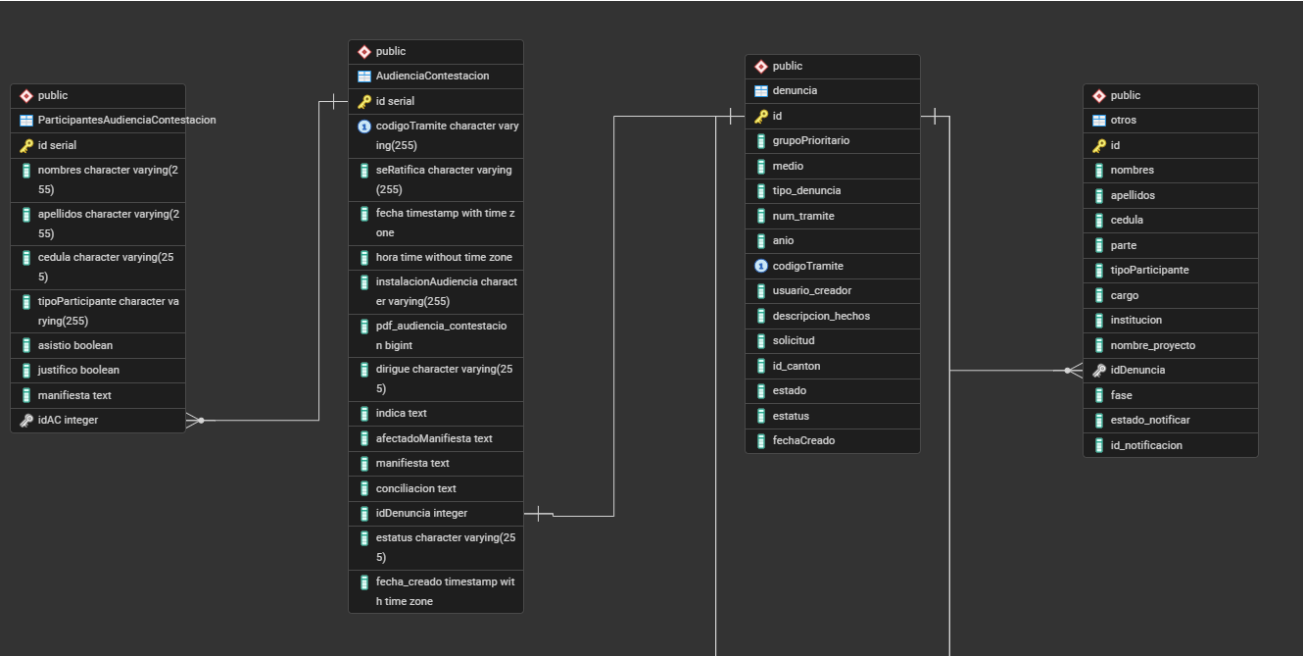


Diagrama relación fase audiencia de pruebas

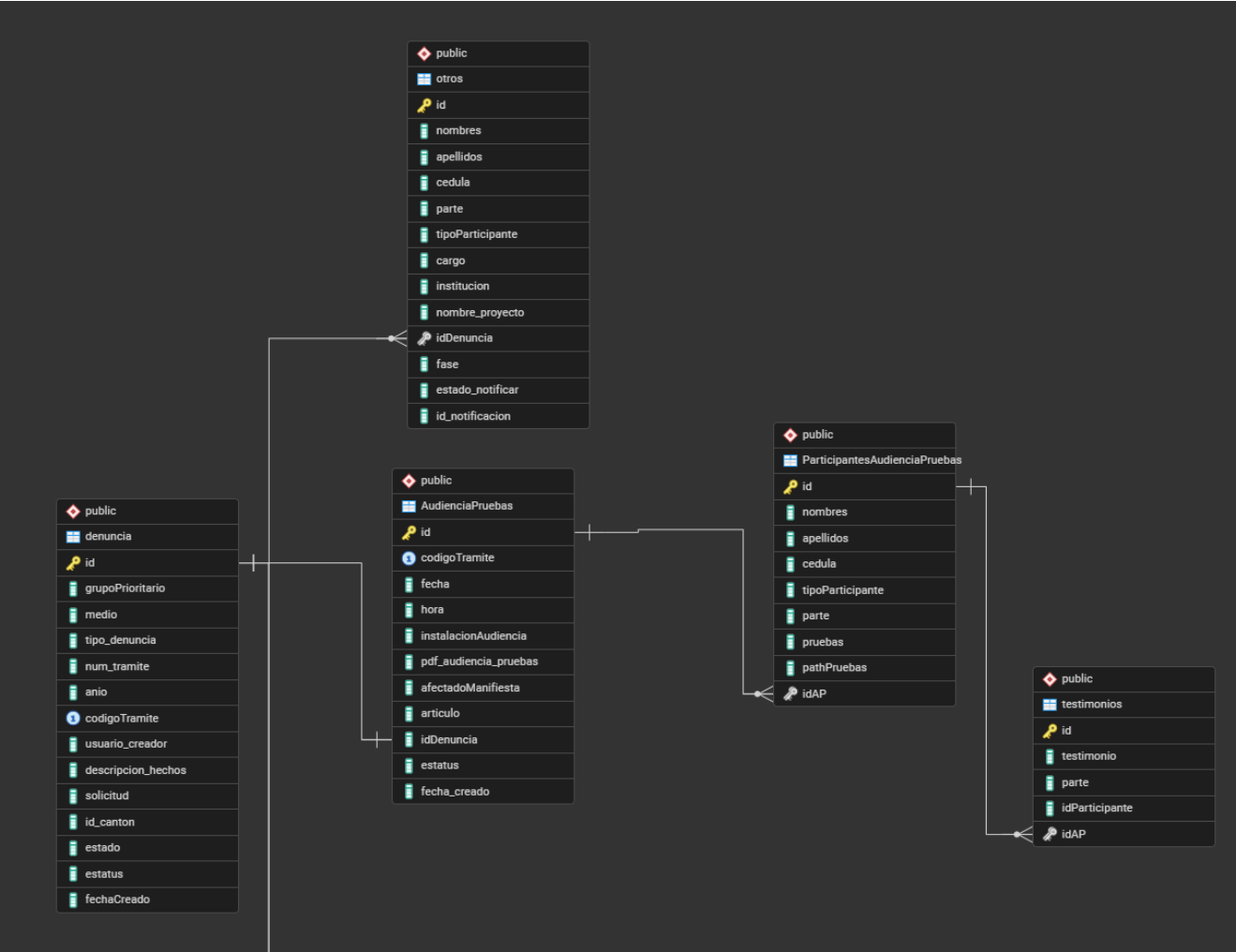


Diagrama entidad relación fase cierre de caso

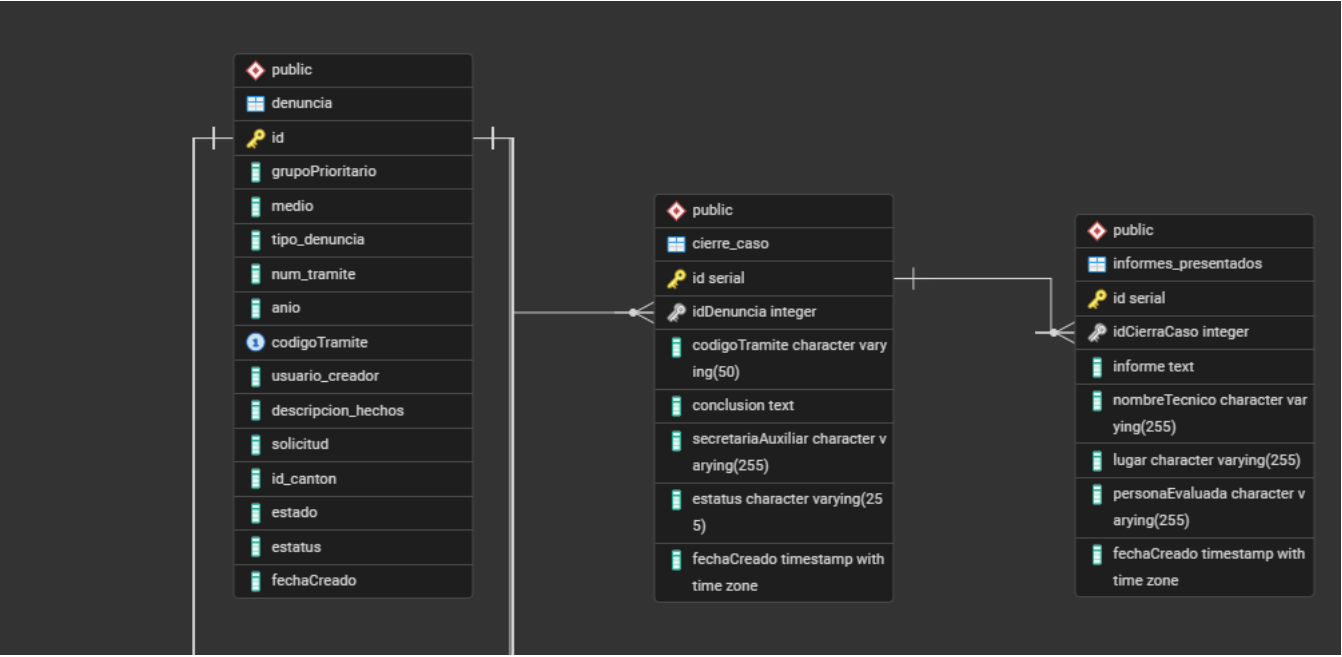


Diagrama fase cumplimiento de medidas denuncia

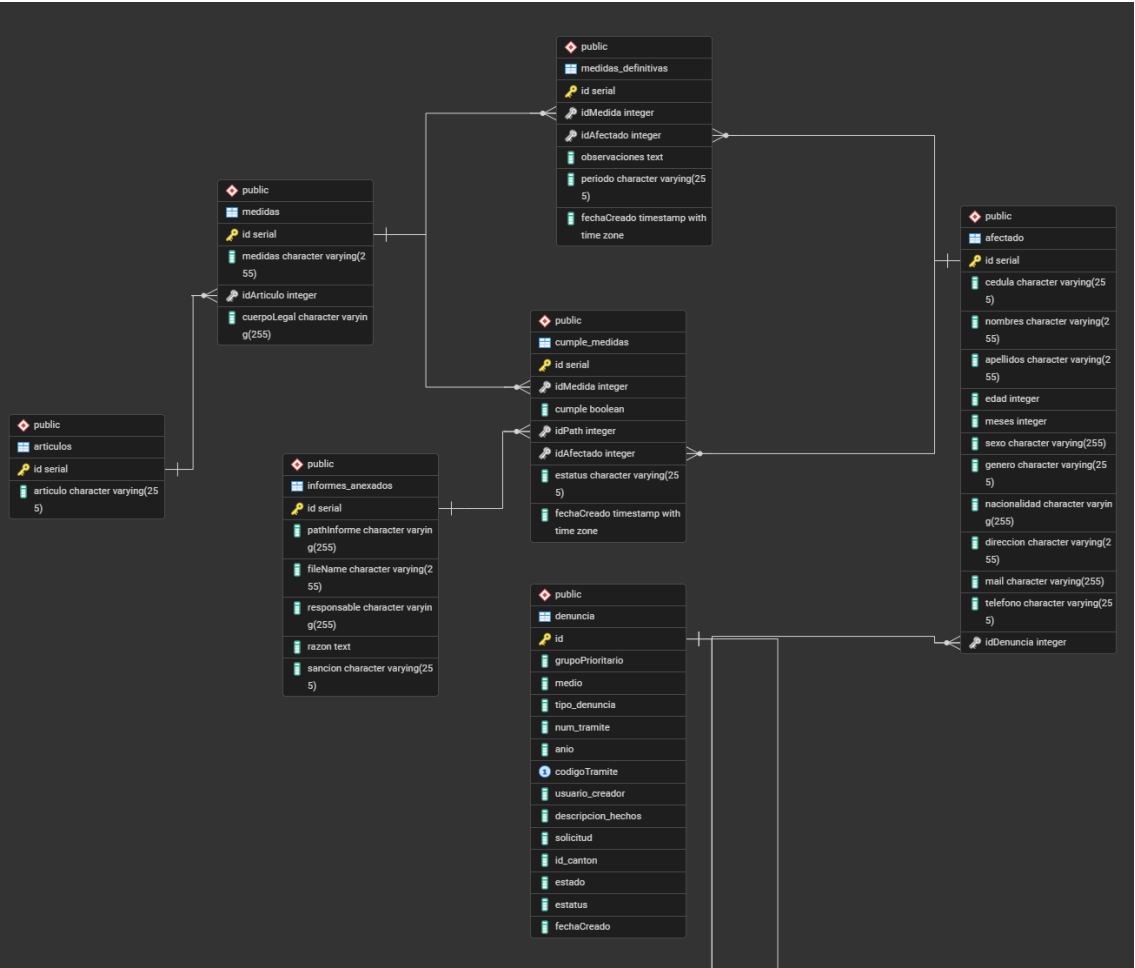


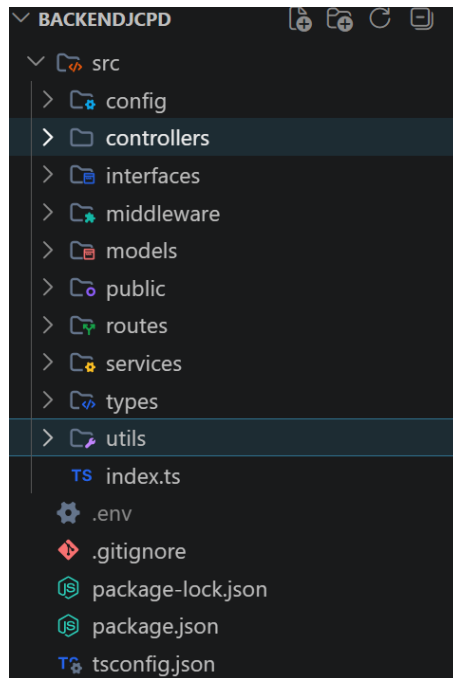
Diagrama relación subida de expedientes



BACKEND

Estructura del código base

El backend sigue una arquitectura multicapa para separar la lógica de enrutamiento de las reglas de negocio puras.



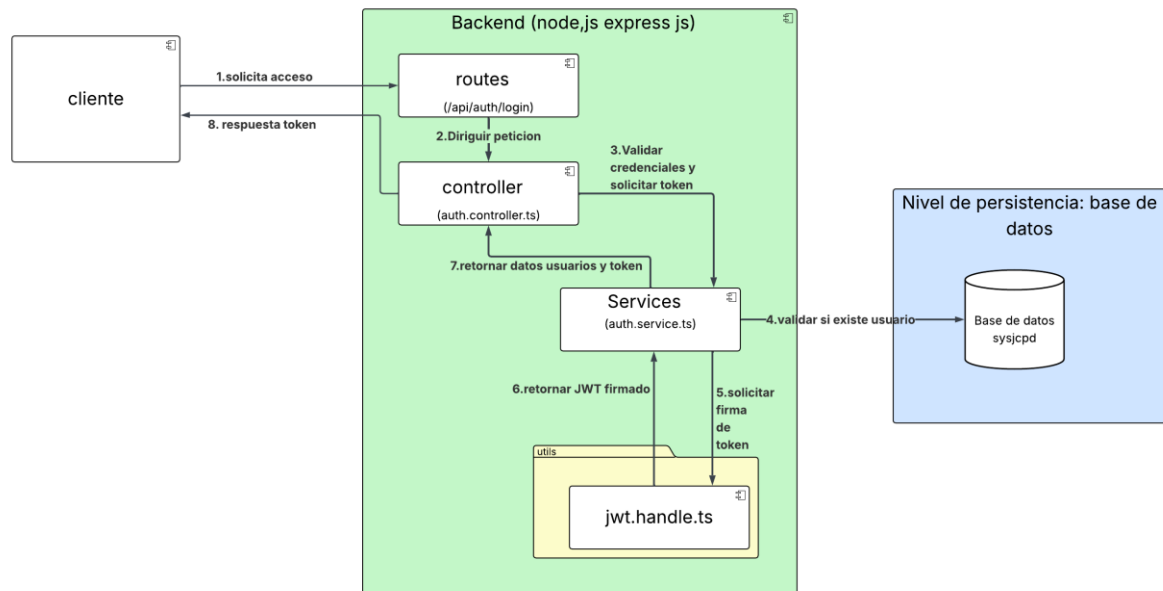
- **Capa de Enrutamiento (/routes):** Actúa como el punto de entrada de la API REST. Define los *endpoints*, aplica los filtros de seguridad (Middlewares) y dirige el tráfico hacia el controlador adecuado.
- **Capa Adaptadora (/controllers):** Su única responsabilidad es manejar la comunicación web. Recibe el objeto *Request* de Angular, extrae el JSON, invoca la lógica de negocio y devuelve una respuesta estandarizada (*Response*).
- **Capa de Lógica de Negocio (/services):** Es el núcleo central de la aplicación. Aquí residen todas las reglas operativas, validaciones legales y procesos propios del control de expedientes de la Junta.
- **Capa de Persistencia (/models):** Contiene la definición estricta de los esquemas de datos.
- **Útil:** Funciones transversales y auxiliares (Helpers) contiene
 - Error.handle.ts
 - Jwt.handle.ts

dependencias

Seguridad

El sistema implementa un modelo de seguridad JSON Web Tokens (JWT) para gestionar la autenticación y autorización del personal de la JCPD.

A continuación, se ilustra el flujo de una petición de inicio de sesión.



Para garantizar la protección de los *endpoints*, toda ruta privada del sistema exige validación estricta. El cliente (Angular) debe adjuntar el token activo en cada petición HTTP de forma obligatoria. Adhiriéndose a los estándares web, este token no viaja en el cuerpo del mensaje, sino que debe enviarse en las cabeceras de la petición bajo el siguiente formato:

Authorization: Bearer <token>

Reglas de Negocio

RN-01: Al registrar una denuncia, el sistema debe autogenerar un código alfanumérico único con el formato [Secuencial de 4 dígitos]-JCPD-[Cantón]-[Año]-[Grupo Prioritario]

RN-02: la denuncia no se puede editar si ya existe una avocatoria

RN-03: la avocatoria no se puede editar si ya existe al menos una notificación

RN-04: la audiencia de contestación no se puede editar si ya existe una audiencia de pruebas o resolución

RN-05: la audiencia de pruebas no se puede editar si ya existe una resolución

RN-06: la resolución no se puede editar si ya existe al menos un cumplimiento de medidas.

RN-07: una vez hecha el cierre del caso la denuncia pasa a estado finalizada

RN-08: debe existir afectados asociados a la denuncia para poder asignar medidas de protección

RN-09: el afectado debe tener vinculado medidas definitivas para poder realizar el cumplimiento de medidas

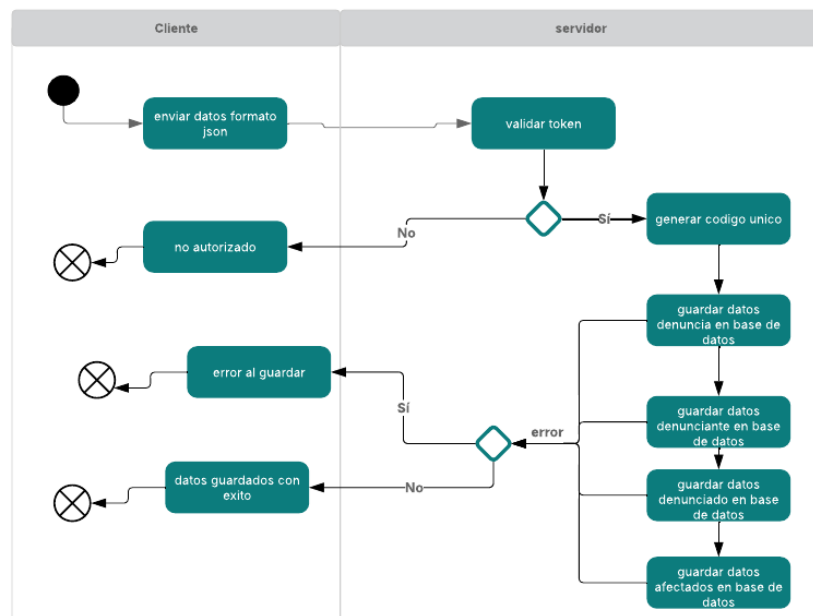
RN-10: los usuarios registrados con su respectivo cantón, solo podrá ver denuncias pertenecientes a el mismo cantón

RN-11: para finalizar el caso todas las medidas dictadas a un afectado deben tener su respectivo seguimiento

Flujo de procesamiento y especificaciones de endpoints

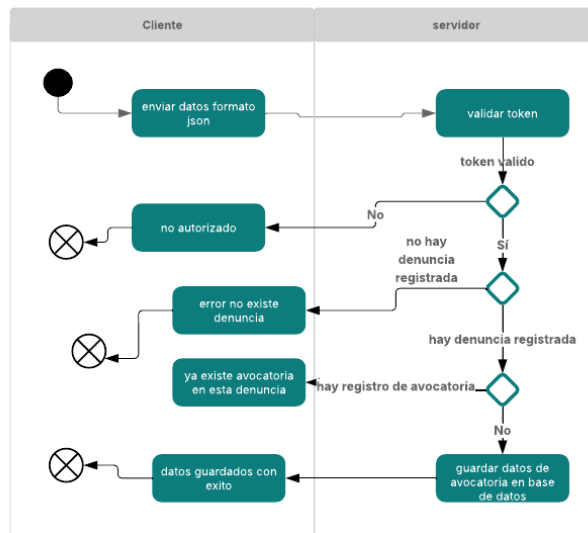
Crear denuncia

Endpoint: POST /api/denuncias/registrar-denuncia



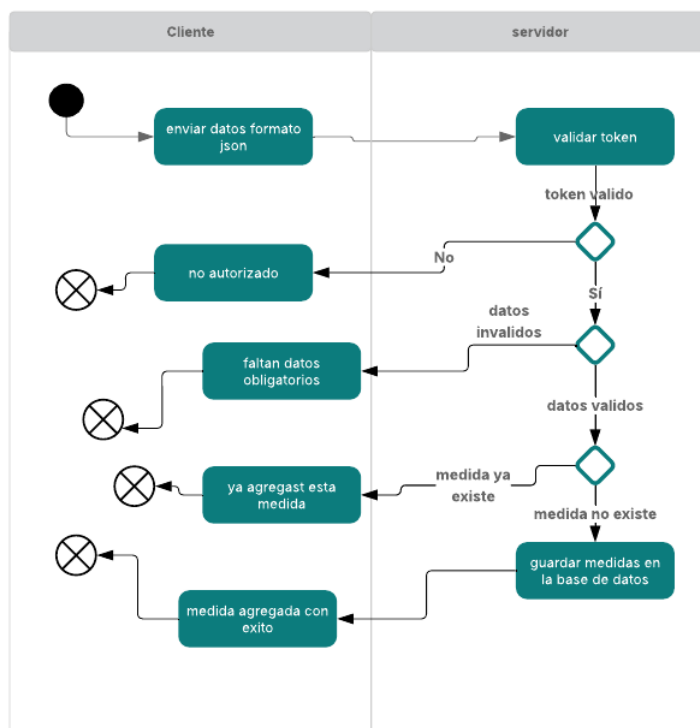
Crear avocatoria

Endpoint: POST /api/avocatoria /registrar-denuncia



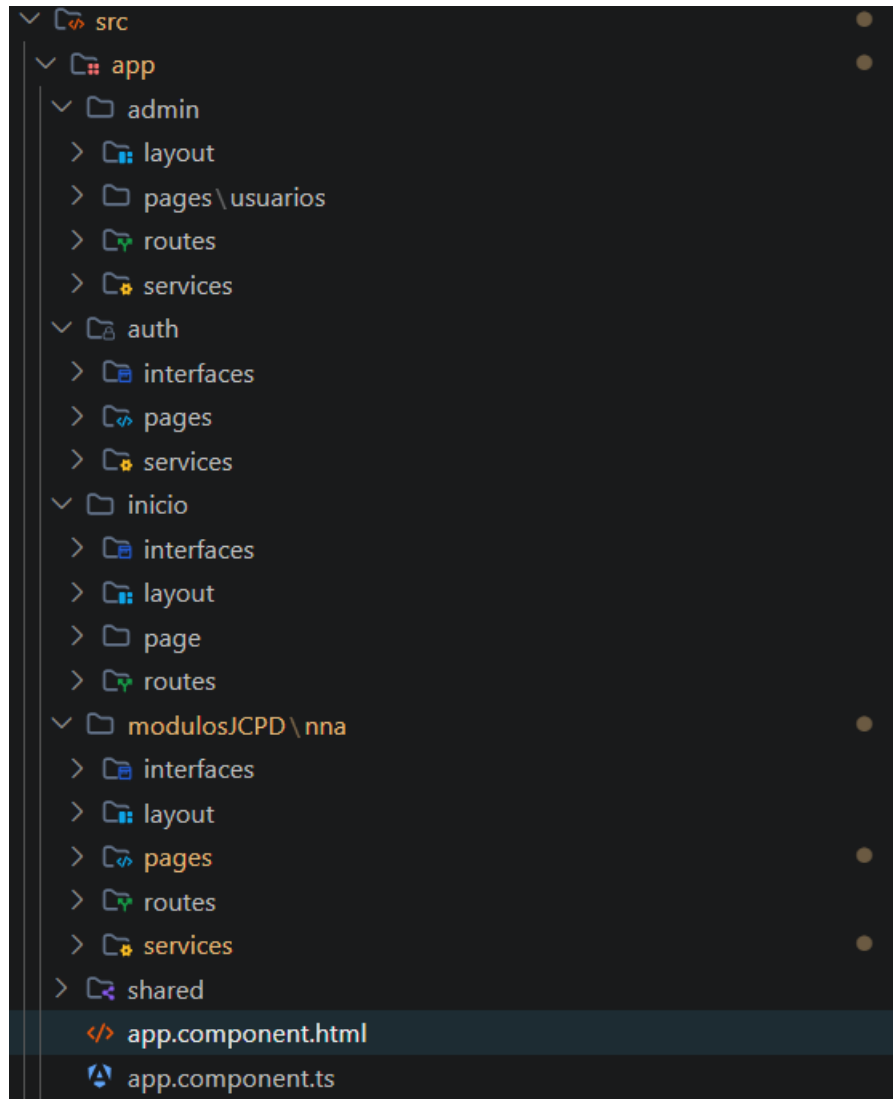
Agregar medidas de protección emergentes

Endpoints: POST /api/medidas/agregar-medidas-emergentes



FRONTEND

Estructura código base



El proyecto frontend, desarrollado en Angular, está organizado mediante una estructura modular que separa las responsabilidades de la interfaz de usuario, la lógica de negocio y las reglas de enrutamiento. A continuación, se detalla la organización principal dentro del directorio src/app:

- **admin/**: Módulo encargado de la gestión de usuarios del sistema.
 - **layout/**: Contiene los componentes estructurales (como barras de navegación o menús laterales) exclusivos para el panel de administración.

- **pages/usuarios/**: Vistas dedicados a la creación, edición y visualización de los usuarios del sistema.
 - **routes/**: Definición del enrutamiento interno (hijo) para las pantallas de administración.
 - **services/**: Lógica de comunicación con la API REST para la gestión de usuarios.
- **auth/**: Módulo de seguridad y control de acceso.
 - **interfaces/**: Definición estricta de tipos de datos (TypeScript) para las credenciales y el token JWT.
 - **pages/**: Vistas de autenticación (pantalla de inicio de sesión).
 - **services/**: Servicios dedicados a gestionar el estado de la sesión y la comunicación con los *endpoints* de autenticación del backend.
- **inicio/**: Módulo de bienvenida o panel de control principal (*Dashboard*) al que accede el usuario tras iniciar sesión exitosamente. Contiene su propio layout, pages, interfaces y routes para mantener su funcionalidad aislada.
- **modulosJCPD/nna/**: Es el módulo central del negocio (Juntas Cantonales de Protección de Derechos).
 - Gestiona todo el ciclo de vida de los expedientes y las diferentes fases de la denuncia.
 - Agrupa sus propias interfaces (modelos de datos legales), layout (estructura visual del expediente), pages (vistas de cada fase procesal), routes y services (comunicación con la base de datos de PostgreSQL a través del backend).
- **shared/**: Carpeta transversal que aloja elementos compartidos por toda la aplicación. Aquí residen componentes reutilizables (ej. modales, botones, alertas), *Guards* de protección de rutas y los *Interceptors* HTTP encargados de inyectar el token de autorización.
- **Archivos Raíz (app.component.ts / app.component.html)**: Actúan como el contenedor principal de la aplicación albergando el <router-outlet> global que

intercambia los módulos mencionados anteriormente mediante *Lazy Loading* (carga diferida).

Diagrama de jerarquía

enrutamiento jerárquico del frontend. El componente raíz (AppComponent) utiliza un enrutador principal (<router-outlet>) para delegar la carga de los módulos funcionales según la ruta solicitada (Lazy Loading). Para mantener una interfaz coherente, cada módulo implementa un patrón 'Layout', donde un componente estructural define el marco visual (menús, cabeceras) y utiliza un <router-outlet> secundario para inyectar dinámicamente los componentes de página (pages) específicos de cada fase o vista.

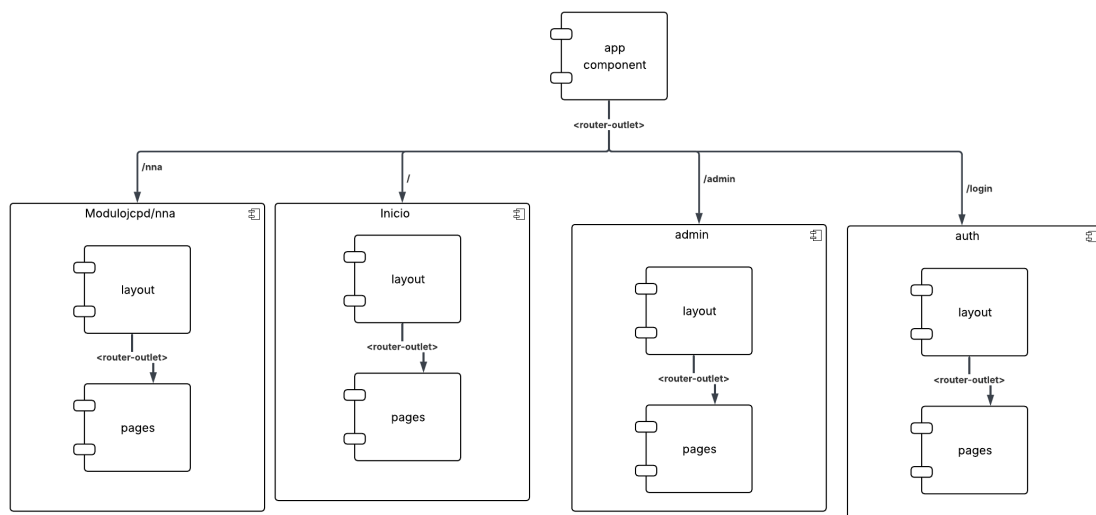


Diagrama de flujos formulario

Diagrama audiencia contestación

